

Microprocessors | HW3 (Reptile + Bird)

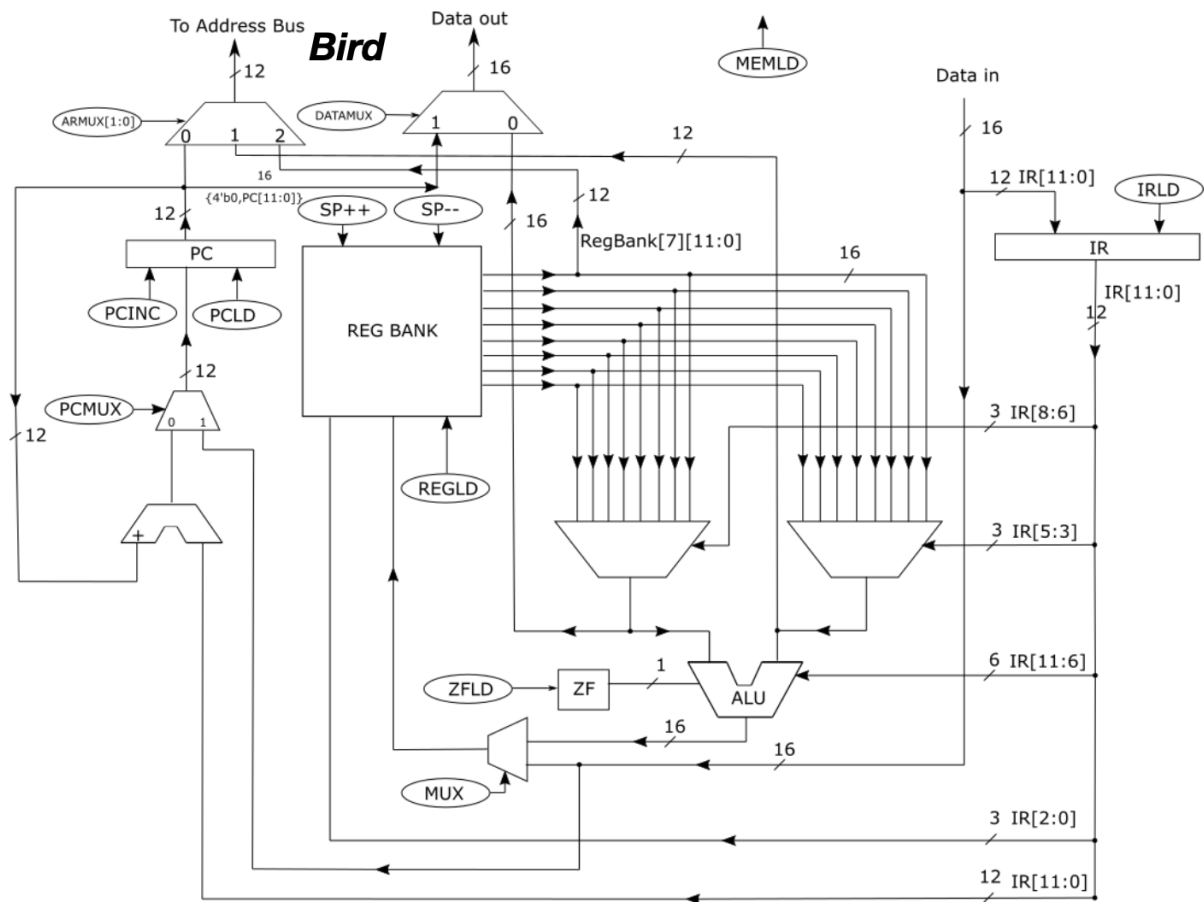
☰ Tags	Design Hardware Microprocessors
▼ Category	Education
🕒 Created time	@November 24, 2024 8:24 AM

✓ CSE4117 Microp_ HW2

Instruction Set

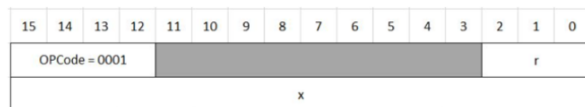
- From **Frog** we have: ALU operations + LDI
- **Reptile** added: LD, ST, JMP, JZ
- **Bird** added: PUSH, POP, CALL, RET

Design



Instruction Formats

Machine Code Format for LDI
LDI rx



Machine Code Format for INC and DEC
INC r1



Machine Code Format for MOV and NOT
MOV r1 r2



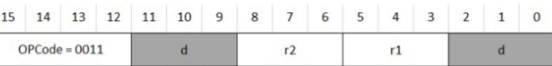
Machine Code Format for ADD, SUB, AND, OR, XOR
ADD r1 r2 r3



Machine Code Format for LD
LD r1 r2



Machine Code Format for ST
ST r1 r2



ALU, LDI, LD, ST Instructions

- JMP instruction is unconditional: it always jumps to the given target location within the code

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCode = 0101				x											

- JZ instruction is conditional: it jumps to the given target location within the code only if the result of the last ALU instruction is zero. Otherwise, the next instruction in the program is executed.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCode = 0100				x											

JMP, JZ instructions

PUSH Instruction

Assembly format: Push r

Writes the contents of the register r into the memory location pointed by the SP. Then decrements SP by one.

Machine code format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCode = 1000				d			r		d						

POP Instruction

Assembly format: Pop r

Pop increments SP by 1, then transfers the content of the memory location pointed by SP into register r.

Machine code format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCode = 1001				d									r		

CALL Instruction

Assembly format: Call x

Pushes the content of the Program Counter (PC) into the stack and jumps to memory address PC+x, i.e, loads the number PC+x into PC. ($PC \leftarrow PC+x$). Recall that PC is already incremented when we start to execute the Call instruction.

Machine code format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCode = 1010				x											

RET Instruction

Assembly format: Ret

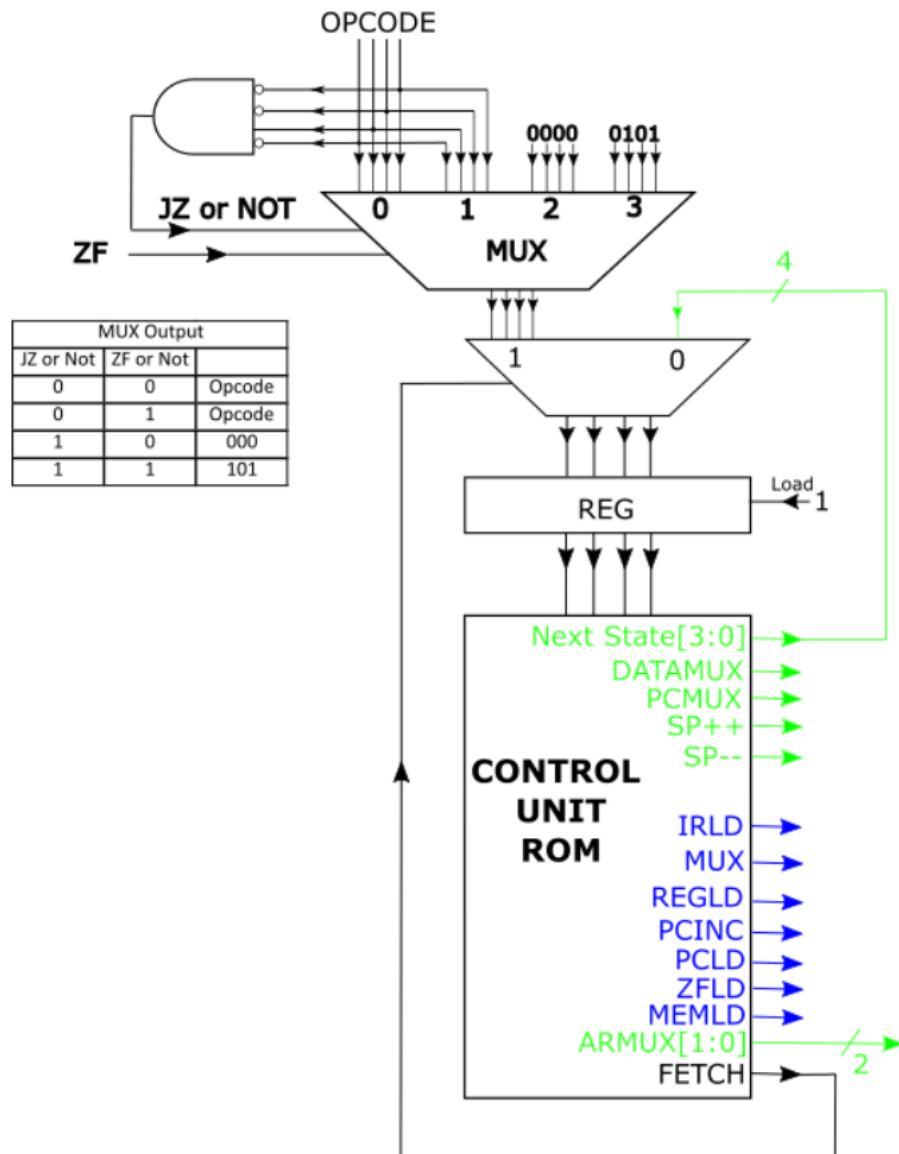
Pops the top of the stack into PC.

Machine code format:

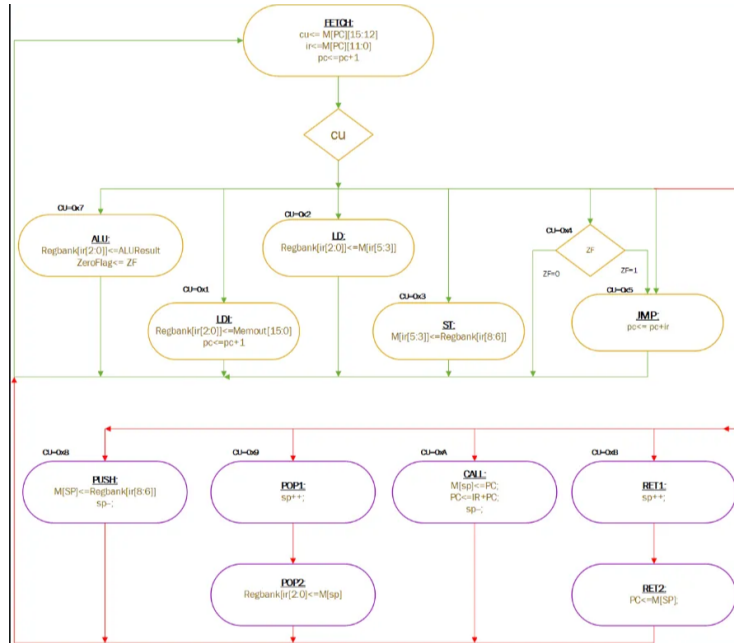
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCode = 1011				d											

PUSH, POP, CALL, RET Instructions

Control Unit



FSM



Microcode

Address	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D[3:0]	HEX CONVERSION
	FETCH	DATAMUX	PCMUX	SP++	SP--	PCLD	PCINC	ARMUX[1]	ARMUX[0]	REGLD	ZFLD	MUX	MEMLD	IRLD	Next State[3:0]	
0	1						1							1	0X0	0x2810
1							1			1		1			0X0	0x940
2									1	1		1			0X0	0x340
3									1				1		0X0	0x220
4															0X0	0x0
5						1									0X0	0x1000
6															0X0	0x0
7										1	1				0X0	0x180
8					1			1					1		0X0	0x240
9				1											0xC	0x400C
A		1			1	1		1					1		0x0	0x13420
B				1											0xD	0x400D
C								1		1		1			0x0	0x540
D			1			1		1							0X0	0x9440

First: 0x20810, Last: 0x9400

Assembler

- 2 noktalıları label tablosuna ekle (eg. LABEL:)
- Diyelim ki jmp geldi, nasıl next state belirlenecek?
- Global current state tutulmalı
- PC + 4 yapmalısın eğer 5 artmalıysa çünkü PC + 1 olacak zaten

- LDI varsa o arada, +1 eklenmeli next adres'i immediate olduğu için

Testing

Based on the instruction format examples, here are examples for all the instructions:

Memory Instructions:

- LDI r1 0x10
0001 000 000 000 001 = 1001 // Load immediate value 0x10 into r1
- LD 1 2
0010 000 000 010 001 = 2011 // Load value from memory address in r2 into r1
- ST 1 2
0011 000 010 001 000 = 3088 // Store value from r1 to memory address in r2

Jump Instructions:

- JZ 0x10
0100 000 000 010 000 = 4010 // Jump if zero flag is set
- JMP 0x20
0101 000 000 100 000 = 5020 // Unconditional jump

ALU Instructions:

- ADD 1 2 3
0111 000 010 011 001 = 7099 // Add r1 and r2
- SUB 3 2 1
0111 001 010 001 011 = 728B // Subtract r2 from r1
- AND 2 4 6
0111 010 100 110 010 = 7532 // Logical AND
- OR 5 4 3
0111 011 100 011 101 = 771D // Logical OR
- XOR 3 2 5
0111 100 010 101 011 = 78AB // Logical XOR
- NOT 1 2
0111 111 000 010 001 = 7E11 // Logical NOT

- MOV 1 2
0111 111 001 010 001 = 7E51 // Move value from r2 to r1
- INC 1
0111 111 010 001 001 = 7E89 // Increment r1
- DEC 1
0111 111 011 001 001 = 7EC9 // Decrement r1

Stack Instructions:

- PUSH 1
1000 000 001 000 000 = 8040 // Push r1 onto stack
- POP 1
1001 000 000 000 001 = 9001 // Pop from stack into r1

Subroutine Instructions:

- CALL 0x30
1010 000 000 110 000 = A030 // Call subroutine
- RET
1011 000 000 000 000 = B000 // Return from subroutine

The format follows the patterns shown in the LDI and LD examples, using 16-bit instruction encoding.